Programming Fundamentals
First Assignment for BS(CS)-1F

General Instructions

- Name C++ files as l26xxxx_q_no.cpp using your roll number for xxxx.
- Understanding the question is part of assignment. Try to think logically, and attempt yourself.
- Make a scanned pdf of flowchart diagrams and pseudocode named as l26xxxx_Algo.pdf
- Submit a .zip file named l26xxxx.zip containing all C++ files and pdf.
- Follow input/output formats if given in sample of a question.
- Use proper variable names, correct indentation, and comments for readability.
- You can comment using double slash //
- Contact the section TA if any one of the problems need clarification/correction

Strictly Avoid Plagiarism

Question 1: The Bacterial Outbreak

A bacteria culture doubles in size every fixed number of minutes. A lab assistant starts with a certain number of bacteria and leaves the sample undisturbed for exactly 3 doubling cycles. You need to work out how much time has passed and how large the population has grown. The population grows fast, so think carefully about what variable type can safely hold the final answer.

Input:

- Starting population, p0 (a whole number, e.g. 10)
- Time taken for one doubling cycle, in minutes, t (e.g. 25)

What you must produce:

- Compute the total elapsed time for 3 doubling cycles (in minutes), then convert it into whole hours and remaining minutes.
- Compute the final population after 3 doublings. Since the population doubles each cycle, this means that you will multiply p0 by 2, three times in total.
- Print the elapsed time as hours and minutes, and the final population, both clearly labelled.
- Draw a flowchart on paper, and then code C++ program, that reads p0 and t.

Sample Input Output
Enter p0: 10
Enter t: 25
Elapsed Time = 1 hour 15 minutes
Final Population = 80

Question 2: The ATM Cash Dispenser

An ATM needs to dispense a requested amount of cash using the fewest possible notes, using only notes of Rs. 5000, Rs. 1000, Rs. 100 and Rs. 10. You may assume the requested amount is always a multiple of 10, so it can always be dispensed exactly with nothing left over.

Input:

- The amount to withdraw, amount (a whole number that is always a multiple of 10, e.g. 8760)

What you must produce:

- Work out how many Rs. 5000 notes are needed, and what amount is left over after removing them.
- Using what is left over, work out how many Rs. 1000 notes are needed, and what remains after that.
- Continue the same idea for Rs. 100 notes, then finally Rs. 10 notes, each time working from the leftover amount of the previous step.
- Print the number of notes of each category.

Draw a flowchart on paper, and then code C++ program that reads amount.

Sample Input Output

Enter amount = 8760

Rs. 5000 notes: 1
Rs. 1000 notes: 3
Rs. 100 notes: 7
Rs. 10 notes: 6

Question 3: The Relay Race Analyst

A school relay team has three runners. Each runner covers a different distance in a different amount of time. The coach wants the team's overall average speed for the whole race, and here is the trap: the overall average speed is NOT the average of the three individual speeds. It is the total distance covered by the team divided by the total time taken.

Input:

- Distance and time for runner 1: d1 (metres), t1 (seconds), e.g. 100, 12
- Distance and time for runner 2: d2 (metres), t2 (seconds), e.g. 100, 11
- Distance and time for runner 3: d3 (metres), t3 (seconds), e.g. 100, 13

What you must produce:

- Compute the total distance covered (sum of d1, d2, d3) and the total time taken (sum of t1, t2, t3).
- Compute the team's overall average speed as total distance divided by total time (in metres per second).
- Separately, compute runner 2's own individual speed (d2 divided by t2).
- Compute how much faster or slower runner 2 was compared to the team's overall average speed (runner 2's speed minus the overall average speed, this can be positive or negative, just print it as is).
- Print the total distance, total time, overall average speed, runner 2's speed, and the speed difference.

Draw a flowchart on paper, and then code C++ program that reads all six values above.

Sample Input Output

```
Enter d1:100
Enter t1: 12
Enter d2: 100
Enter t2: 11
Enter d3: 100
Enter t3: 13
Total Distance: 300
Total Time: 36
Overall Average Speed = 8.33 m/s
Runner 2 Speed = 9.09 m/s
Speed Difference = 0.76 m/s
```

Question 4: The ID Verification System

Fast NUCES FLEX assigns every student a 4-digit ID. To catch typing mistakes, it generates a one-digit verification code from the ID using a weighted formula, similar to how credit card numbers are checked. Before computing anything, you first need to split the ID into its individual digits, without ever treating it as text.

Input:

- A 4-digit student ID, id (e.g. 6432)

What you must produce:

- Extract its four digits into four separate variables: d1 (thousands), d2 (hundreds), d3 (tens), d4 (units) using only / and % operators.
- Compute a verification code using this formula: code = (d1*4 + d2*3 + d3*2 + d4*1) % 9
- Compute a "mirror ID", formed by writing the same four digits in reverse order and recombining them into a single 4-digit number (e.g. 6432 becomes 2346) (Hint: simple maths would be applied here)
- Print the four digits, the verification code, and the mirror ID
- Draw a flowchart on paper, and then code C++ program that reads the 4-digit ID.

Sample Input Output

Enter id: 4827
Digits: 4 8 2 7
Verification Code: 6
MirrorID: 7284

Question 5: The Garden Designer

An architect is planning a rectangular garden. Running all the way around the inside edge of the garden is a uniform paved walking path of a certain width. Once the path is taken out, whatever space is left in the middle,, becomes the lawn. The architect needs three numbers to prepare a budget: how much fencing is needed for the outer boundary, how much paving is needed for the path, and how much turf is needed for the lawn, each priced differently per unit, and finally the total project cost.

Input:

- Length and width of the garden, length and width (in metres, e.g. 20, 15)
- Width of the path running around the inside edge, pathWidth (in metres, e.g. 2)
- Cost per metre of fencing, fenceCost (e.g. 150)
- Cost per square metre of paving, paveCost (e.g. 80)
- Cost per square metre of turf, turfCost (e.g. 40)

What you must produce:

- Compute the total garden area (length * width).
- Compute the lawn's dimensions by removing the path from both sides of each edge (the lawn's length is the garden's length minus twice the path width, and similarly for its width), then compute the lawn area.
- Compute the path area as the garden area minus the lawn area.
- Compute the outer perimeter of the garden (for fencing): 2 times the sum of length and width.
- Compute the total cost as: (perimeter × fenceCost) + (path area × paveCost) + (lawn area × turfCost).
- Print the garden area, lawn area, path area, perimeter, and total cost, all clearly labelled.
- Draw a flowchart on paper, and then code C++ program that reads all six values above.

Sample Input Output

Enter length: 20
Enter width: 15
Enter pathWidth: 2
Enter fenceCost: 150
Enter paveCost: 80
Enter turfCost: 40
Garden Area = 300
Lawn Area = 176
Path Area = 124
Perimeter = 70
Total Cost = 27460

Q 6: Time Conversion

Write a program that inputs a total number of seconds and converts it into:

Hours
Minutes
Remaining seconds

For example, 3672 seconds should become:
1 hour, 1 minute, 12 seconds

Use arithmetic operators and integer division carefully.

Q 7: Three-Digit Number Analysis

Write a program that inputs a three-digit positive integer and extracts:

The hundreds digit
The tens digit
The ones digit
The sum of its digits
You must use integer division / and remainder %.

Example
Enter a three-digit number: 472

Hundreds digit: 4
Tens digit: 7
Ones digit: 2
Sum of digits: 13
Reversed number: 274

Conditions
Assume that the user enters a three-digit number.
Do not convert the number into a string.
Do not use loops or conditional statements.
No arrays

Q 8: Digital Clock Calculation

Input the current time and a number of seconds to add to it.

Input:
Current hour
Current minute
Current second
Number of seconds to add

Calculate the new time after adding the given number of seconds.

Assume the clock uses a 24-hour format.

Example:
Enter hour: 23
Enter minute: 58
Enter second: 45
Enter seconds to add: 130

New time: 00:00:55

Suggested Approach
First convert the current time into total seconds:

Add the extra seconds, then use % to keep the time within one day:

Then calculate the new:

Hour
Minute
Second

Q 9: Compound Interest Calculator

Write a C++ program that calculates the final amount and compound interest for a bank investment.

Use the formula:

A = P(1+r/n)^{nt}

Where:

P = initial principal amount
r = annual interest rate as a decimal
n = number of times interest is compounded per year
t = number of years
A = final amount

The compound interest is calculated as:

Compound Interest = A-P

Input

Your program should ask the user for:

- Initial amount
- Annual interest rate in percentage
- Number of times interest is compounded per year
- Number of years

Example

Enter initial amount: 10000
Enter annual interest rate: 5
Enter compounds per year: 4
Enter number of years: 3

Final amount: 11607.55
Compound interest: 1607.55

Q 10: Cricket Chirp Temperature Calculator

Task:
Create a Cricket Chirp Temperature Calculator. The program should ask the user to enter the number of cricket chirps counted in one minute. Use the given formula to estimate the temperature in degrees Fahrenheit, then display the calculated temperature.

Formula:

Temperature (°F) = 50 + (ChirpsPerMinute - 40) / 4

Input Requirement:
The program must take the number of cricket chirps per minute as input from the user.

Example Input:

Chirps per minute = 80

Required Output:

Estimated Temperature: 60°F

Optional

Q 11: Student Average Calculator

Task:
Create a Student Average Calculator. The program should ask the user to enter the marks obtained in three subjects as integer values. Use these input values to calculate the student's average and display the result as a decimal value. Make sure the calculation does not lose the fractional part because of integer division.

Formula:

Average = (Math + Physics + Programming) / 3

Input Requirement:
The program must take Math, Physics, and Programming marks as input from the user.

Example Input:

Math = 87
Physics = 78
Programming = 92

Required Output:

Average Marks: 85.6667

Q 12. The Kinetic Energy Trap (Focus: Typecasting & Integer Division)

Scenario: A physics engine calculates the kinetic energy of a moving projectile. The user inputs the mass as an integer and velocity as a floating-point number. The formula uses a literal fraction (1/2). You must write the expression exactly as given using 1/2 (not 0.5), forcing you to handle the typecasting in C++ so that the integer division 1/2 does not erroneously evaluate to zero before multiplying with the rest of the variables.

Formula:
KE = 1/2 * m * v * v
Example Input: Mass = 5, Velocity = 3.5
Example Output: Kinetic Energy: 30.625

Q 13. Logical Access Matrix (Focus: Relational & Logical Operators)

Scenario: A server firewall evaluates user access using a rigid matrix. Access is granted (1) or denied (0). An employee is granted access only if their age is 18 or older AND their security clearance level is strictly greater than 3, OR if they possess a master override code exactly equal to 999. Output the boolean result directly to the console.

Formula:
Access = (Age >= 18 && Clearance > 3) || (OverrideCode == 999)
Example Input: Age = 17, Clearance = 5, OverrideCode = 999
Example Output: Access Granted: 1

Q 14. Network Packet Fragmentation (Focus: Modulo Operator)

Scenario: When transmitting data over a network, files are broken into standard-sized chunks called "packets." Given a total file size in bytes and a fixed network packet size, determine exactly how many full packets will be successfully transmitted, and how many bytes will be left over in the final, incomplete packet fragment.

Formula:
FullPackets = TotalBytes / PacketSize
LeftoverBytes = TotalBytes % PacketSize
Example Input: Total Bytes = 14560, Packet Size = 1024
Example Output: Full packets to transmit: 14, Leftover bytes: 224

Q 15. Structural Stress Polynomial (Focus: Operator Precedence & Complex Expressions)

Scenario: An engineering stress-test algorithm relies on evaluating a 4th-degree polynomial to predict the structural failure point of a new composite material. You are tasked with hardcoding this evaluation for a given floating-point value, x. Construct a single, continuous mathematical expression that correctly applies multiplication and addition/subtraction precedence.

Formula:
y = 3 * x * x * x * x - 2 * x * x * x + x - 7
Example Input: x = 2.5
Example Output: Stress factor y: 81.4375

Q 16. Digital Clock Converter (Focus: Integer Division & Modulo Operator)

Scenario: Create a Digital Clock Converter. The program should ask the user to enter a total number of elapsed seconds as an integer. Convert this value into hours, minutes, and remaining seconds using integer division and the modulo operator. Display all three values separately.

Formula:
Hours = TotalSeconds / 3600
RemainingSeconds = TotalSeconds % 3600
Minutes = RemainingSeconds / 60
Seconds = RemainingSeconds % 60
Example Input: Total seconds = 7384
Example Output: Hours: 2, Minutes: 3, Seconds: 4

Q 17. 3D Distance Calculator (Focus: Operator Precedence & Arithmetic Expressions)

Scenario: Create a 3D Distance Calculator. The program should ask the user to enter the x, y, and z coordinates of two points. Calculate the squared distance between the two points. Do not use the pow() function or any external math library.

Formula:
D2 = (x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1) + (z2 - z1) * (z2 - z1)
Example Input: x1 = 2, y1 = 3, z1 = 1, x2 = 7, y2 = 5, z2 = 4
Example Output: Squared Distance: 38